

Product Requirements Document: AI Ticket Intake

Product: AI Ticket Intake

Author: Christina Y Zhang

Date: April 2026

Status: Draft v1

Contents

- [a. Problem Definition](#)
- [b. Customer Segments & Use Cases](#)
- [c. Scope Definition](#)
- [d. User Flow](#)
- [e. Functional Requirements](#)
- [f. Non-Functional Considerations](#)
- [g. Success Metrics](#)
- [h. Risks & Open Questions](#)

a. Problem Definition

Target Users

1. **Ticket Submitter (Customer):** Customers are dissatisfied with the ticket submission process. They find it too complicated and inefficient.
2. **Ticket Handler (Technical Engineer):** Engineers believe the information provided by customers is not sufficient to understand the issue. They often need further clarification and cannot proceed directly with troubleshooting.
3. **Business & Operations Stakeholders:** They care about how fast tickets get handled, whether customers are happy, and how much it's all costing.

4. **Ticket Routing Owners (Scrum Masters / Support Managers):** They're trying to categorize and assign tickets, but the information is inconsistent or incomplete. Tickets end up going to the wrong person, and things get delayed.

Core Pain Points

1. **Users don't know enough about the product to describe their problems well.** They can't identify the issue type, don't know how to articulate it, and they end up leaving out key context even when they try.
2. **The current intake system relies on users typing everything in manually.** No guidance, no auto-complete, no context-aware prompts. So the quality of the ticket depends entirely on how well the user can express themselves.
3. **Tickets come in incomplete, and engineers have to start from scratch.** They can't just jump into troubleshooting. They have to go back and forth with the customer for at least 2-3 rounds before they understand what's actually going on. That adds up. Multiple rounds of clarification drag out handling time, and the customer is waiting the whole time.
4. **Business and operations people don't have visibility into where things are breaking down.** They want to improve efficiency and reduce cost, but they can't pinpoint what's slowing tickets down.
5. **Ticket routing depends on whoever's assigning it.** It's gut feel and keyword guessing, not a system. Misroutes happen a lot.
6. **Senior engineers end up working on tickets that should've been handled at L1.** The description was too vague to route properly, so it bubbles up. That's expensive time on the wrong work.
7. **Tickets come in from portal, email, web forms, phone.** Each channel has a different level of structure. There's no unified way to make sure the quality is consistent.

Why This Problem Matters Now

- **AI is evolving fast.** If we don't bring AI into this workflow, we'll fall behind competitors who do.
- **Customers struggle with the submission process.** They don't have enough product knowledge to assess how serious their problem is, and they shouldn't have to.
- **Market timing:** 85% of MSPs say automation is a must-have (Kaseya 2024 Benchmark).
- **Engineers waste time chasing context.** They go back to customers to rewrite problem descriptions. That's inefficient use of human effort.
- **AI gives leadership visibility.** It can help Business Stakeholders, Operation Managers, and Technical Support Managers get a handle on ticket quality and cost from the top down.

- **AI reads better than most users write.** It can dramatically improve how tickets get filled out, even from vague input.
- **LLM capabilities are production-ready.** Conversational intake is no longer experimental.
- **Back-and-forth is killing handling time.** Multiple rounds of clarification drag out what should be simple tickets.
- **Senior engineers are stuck on the wrong tickets.** Low-quality tickets that shouldn't have reached them are eating expensive time and driving up support costs.

b. Customer Segments & Use Cases

User Segments

1. **Technical Background Customer:** They've used the product before. They know where the problem is. But they find the ticket submission process tedious with too many fields to fill manually. What they need is speed: click a few things, let AI auto-fill the rest, and submit fast.
2. **Non-technical User:** They don't understand the system or the product. They don't know which component their issue belongs to, can't describe the problem clearly, and have no idea how serious it is. They need AI to guide them through it step by step, using plain-language questions to help them articulate what's going on.
3. **MSP / Account Manager:** They're submitting on behalf of multiple customers and managing multiple client environments. They need MSPbots to auto-analyze incoming cases and handle them in bulk, not one at a time.

Note: These user types exist across MSPs of all sizes. In smaller MSPs (1-10 staff), non-technical users make up a larger share because there's no dedicated support coordinator to help. In mid-size MSPs (10-50 staff), the volume is higher and the account manager segment becomes more important.

Primary Ticket Intake Scenarios (10 Use Cases)

Use Case 1: System error, user can't describe it

The system throws an error. The user doesn't know which component is broken and doesn't know how to describe what happened.

AI response: Auto-parse the error context from whatever the user can tell us, and generate a structured description on their behalf.

Use Case 2: Feature not working, unclear if it's a bug or config issue

A feature that used to work isn't behaving right, or something that was recently set up isn't functioning as expected. The user doesn't know if it's a bug, a misconfiguration, or something else entirely.

AI response: Guide the user through questions to supplement the missing info and help determine the type of issue.

Use Case 3: Performance degradation, user unsure of severity

The system is slow, latency is high. The user doesn't know whether this is normal or something they should worry about.

AI response: Assess the severity and provide a recommendation on whether this needs urgent attention or can wait.

Use Case 4: Waiting too long, wants a status update

The user submitted a ticket a while ago and hasn't heard back. They're not sure if anyone is even looking at it.

AI response: Auto-detect how long they've been waiting. Explain the current status: is it being worked on, or no one picked it up yet? Based on the customer's tone and the nature of the issue, suggest whether escalation is appropriate.

Use Case 5: Non-technical user, doesn't know the right terms

The user doesn't have a technical background. They don't know what terminology to use and the information they provide is incomplete.

AI response: Walk them through it step by step with simple questions, building up the ticket as they answer.

Use Case 6: Post-upgrade regression

The user just upgraded to a new product version and suddenly some features aren't working. They don't know if it's a bug or a compatibility issue.

AI response: Detect the version change, cross-reference against known issues and KCS articles, and surface any relevant matches.

Use Case 7: Deployment failure, too many logs to read

The user's product installation or deployment failed partway through. The log output is massive and they can't figure out where things went wrong.

AI response: Parse the logs and auto-extract the key error information so they don't have to dig through thousands of lines.

Use Case 8: Access or authentication problem

The user is getting blocked by an access or authentication issue. They see "Access Denied" or

"Forbidden," or their permissions have changed and they can no longer reach what they need.
AI response: Classify it as an auth/RBAC issue and guide the user to provide the necessary details (what they see, what changed, what they're trying to access).

Use Case 9: Multi-component issue, user doesn't know who should handle it

The problem spans multiple components, like network and storage, or monitoring and logging. The user has no idea which team should be looking at this.
AI response: Route the case to the right SE team, break down the problem structure into its parts, or pass it to the SE manager to make the call.

Use Case 10: System down, core business impacted

The whole system is down and it's affecting core business operations. Everything is urgent.
AI response: Auto-mark as Severity 1 and trigger escalation immediately.

c. Scope Definition

v1 Scope (In)

Three ways to submit a ticket. The user picks what works for them:

Mode	How it works	Who it's for
Type it out	User describes the problem in their own words, AI analyzes and fills in the rest	People who know what's wrong and just want to get it done fast
Talk it through (chat)	AI asks questions one at a time, user clicks answers or types short replies	People who aren't sure how to describe the problem, or non-technical users
Call AI support (voice)	AI talks to the user like a real person. User speaks, AI listens, and the transcript is editable before submitting	People who'd rather just talk than type. Also good for accessibility

What AI does across all three modes:

- **Suggests the category and subcategory** based on what the user said, and shows why. Like "We put this under Email > Client App because you mentioned Outlook crashing."

- **Suggests a priority level** based on how urgent and impactful it sounds, with a short explanation so the user understands the reasoning.
- **Generates a summary and structured description.** When the user can't describe the problem well (UC1, UC7), AI takes whatever they said and puts together something an engineer can actually work with.
- **Fixes typos before analyzing.** If the user misspells "netwrok" or "permission," AI corrects it and shows what it changed. The corrected version is what gets analyzed, but the original is preserved.
- **Pops up smart suggestions while you type.** If someone types "I don't know which component," a list of common components shows up for them to pick from. Same thing for errors, login issues, slow systems, etc.
- **Catches missing info.** If required fields or critical context are missing, AI asks specific follow-up questions instead of just saying "please fill in all fields."
- **Helps figure out the root cause.** When someone says something vague like "it's not working," AI asks targeted questions to narrow it down: is it an access issue, a config problem, or a bug?
- **Assesses how serious it is.** For performance complaints or outage reports, AI tells the user whether this looks like an emergency or something that can wait until later today.
- **Checks for duplicate tickets** before submission. Follows a three-tier privacy model: your own tickets show full details, tickets from your company show anonymized info, tickets from other companies are never exposed.
- **Looks up existing tickets.** If someone is just asking about a ticket they already submitted, AI finds it and shows the status instead of creating a new one.
- **Matches against known issues and KCS articles.** If the user mentions an upgrade or version change, AI checks whether there's already a known issue or a knowledge article that could solve it without filing a new ticket.
- **Parses logs and errors.** When a user says their deploy failed and there's a wall of logs they can't make sense of (UC7), AI pulls out the key error so they don't have to read through thousands of lines.
- **Breaks down multi-component problems.** When the issue touches network, storage, monitoring all at once (UC9), AI separates it into parts and routes each to the right team. If it's not clear who should own it, it goes to the SE manager.
- **Auto-escalates critical incidents.** If AI picks up that the system is down and the business is affected, it marks it Severity 1 and triggers escalation right away. No waiting. Engineer Manager will review the case and make sure it is real sev1 so can go to support engineer.
- **Nothing gets submitted without the user saying so.** Every AI suggestion is editable. The user always has the final say.

Voice call specifics:

- Pick an accent that feels natural: American, British, Australian, Indian, or Irish. Each one uses a natural voice that sounds like a real person, not a robot.
- The user talks as long as they want. When they're done, they hit "I'm done" and AI processes what they said.
- Before AI does anything with it, the user can edit the transcript. If speech recognition got a word wrong, they fix it right there.
- If the natural voice isn't available for some reason, it falls back to the browser's built-in voice.

v1 Out of Scope

Screenshot / image analysis

We'd need a separate vision API like OpenAI Vision or Google Cloud Vision, which means another API key, another cost line, and latency to manage. Plus we'd have to test it across all kinds of screenshot formats: Slack messages, error popups, monitoring dashboards. It's doable, but it's not what makes or breaks v1.

Multi-language support

Doing this right means having native speakers QA each language, making sure the prompts feel natural and not just Google-translated, and having people on the team who actually know those languages. We're going English-first because that's where MSPbots' market is right now.

Per-MSP customized AI models

Every MSP runs a different stack with different component names and categories. Ideally AI should be trained for each MSP's environment, but in v1 we don't know yet what systems users will report. We need real data first to determine what's worth customizing.

Chat / Teams / Slack intake

It's a totally different UX paradigm. You'd need platform-specific API integrations, bot registration, OAuth flows. That's 3-4 extra weeks of work and a different type of interface to design.

Email intake parsing

Valuable, but hard to do well. Emails come in all kinds of formats, you need auto-reply logic for incomplete ones, and a coordinator has to review the drafts. Too much complexity to pile on top of v1.

RMM alert auto-conversion

Machine-generated tickets and human-submitted tickets are completely different problems. Different data, different context, different routing. Mixing them in would muddy the scope.

Fully automated submission

We're not letting AI submit tickets without the user confirming. Not until we have v1 data showing AI accuracy is consistently above 90%. You have to earn trust before you get that kind of autonomy.

v2 Roadmap

Screenshot-to-ticket

Sometimes it's easier to just screenshot the problem than try to explain it. A user sees their coworker complaining about something in Slack, or gets a weird error popup. They paste that screenshot right into the intake field. AI reads the text out of the image and works with it like the user typed it. Way more accurate than asking someone to remember what an error message said.

Submit on behalf of someone else

This happens all the time: "Hey, Sarah's laptop just blue-screened and she can't get online, can I file a ticket for her?" Right now that's awkward because the ticket ends up under the wrong person. In v2, AI picks up on the "on behalf of" context, keeps track of who's submitting vs. who's actually affected, and links both to the ticket.

AI estimated resolution time

After you submit a ticket, instead of just "Thanks, we got it," AI tells you something like "Tickets like this usually take 2-4 hours." It's a small thing, but it cuts down on the "is anyone even working on this?" follow-ups because people know what to expect.

Multi-language support

Not just for typing, for all three modes. A user in Brazil should be able to call in Portuguese, a user in Mexico types in Spanish, and the technician still sees the ticket in English. The voice call mode would get language-specific speech recognition and TTS voices too. This opens the door for MSPs with branches all over the world.

Future Scope (v3+)

Self-service resolution

Before even creating a ticket, AI asks "Want to try a quick fix first?" and walks the user through something simple like clearing a cache or restarting a service. If it works, no ticket needed. Could cut ticket volume by 10-20%.

Environment auto-collection

If the device has an RMM agent installed, AI pulls in the OS version, disk space, last patch date, recent software installs, all automatically. The technician opens the ticket and everything they'd normally have to ask for is already there.

Smart ticket correlation

Goes beyond "is this a duplicate?" to "is this related to something else going on?" Like if someone reports Outlook crashing and there are three network-instability tickets from the same building in the last hour. Maybe Outlook isn't the real problem.

Suggested resolution for technicians

AI looks at how similar tickets were solved in the past and suggests it: "70% of the time this was fixed by clearing the Outlook cache, 20% needed the add-in reinstalled." Gives the tech a head start instead of diagnosing from scratch.

Trend alerting

When the same kind of ticket starts coming in faster than normal (five VPN issues from Building A in one hour), AI flags it to the ops manager before it becomes a bigger fire.

Ticket quality scoring

Every ticket gets a quality score. How complete is it, is the category right, is the description clear. Lets ops managers see the trend over time and prove the ROI of AI Intake with real numbers.

Chat / Teams / Slack integration

Let people submit tickets right from where they already work, without switching to a separate portal.

Configurable automation level

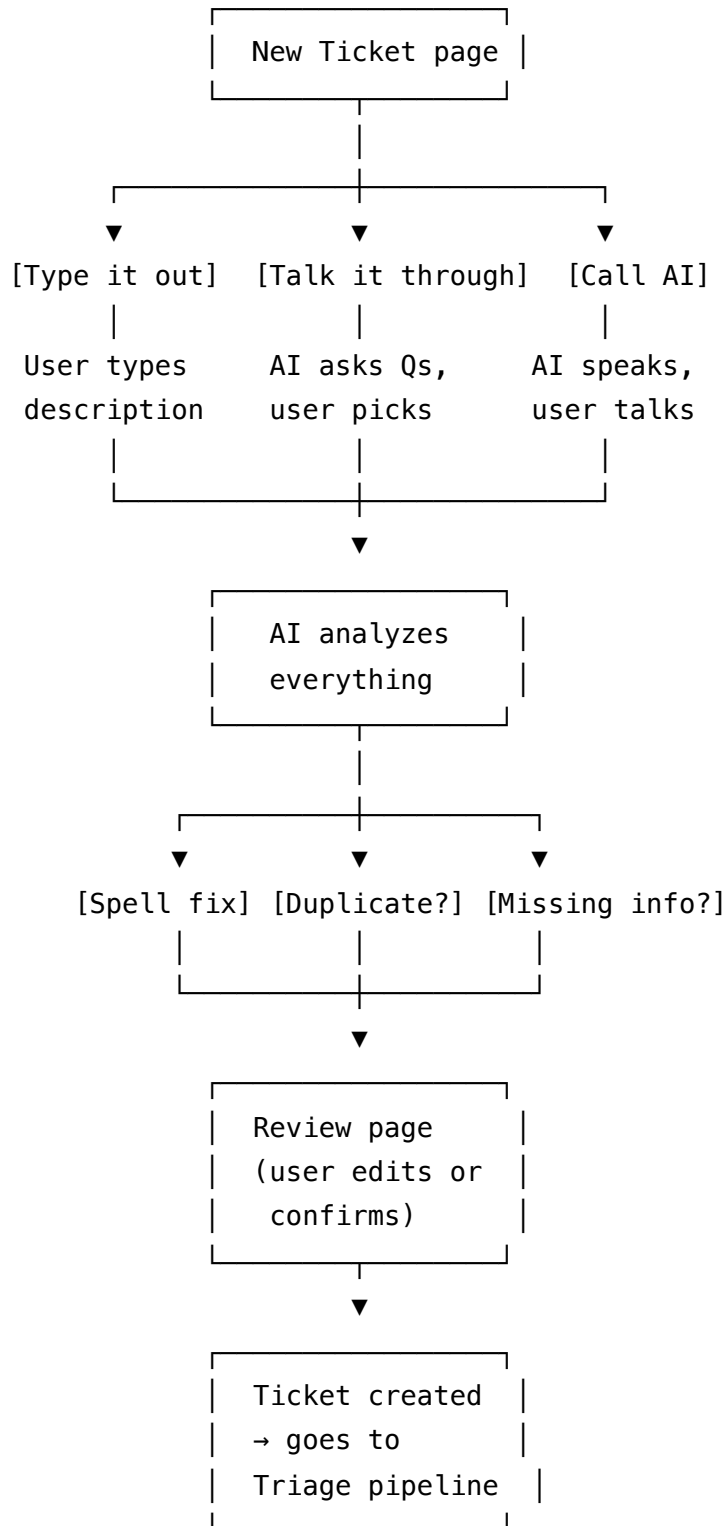
Some MSPs want AI to just suggest; others want it to auto-fill everything; a few might even want auto-submit. Let each MSP dial it to their comfort level.

d. User Flow

The big picture

image.png

User opens the portal and clicks "New Ticket." They see three options:



All three paths end up in the same place. The difference is how the information gets collected.

Path 1: Type it out

1. User types their problem in a free-text box.

2. While they're typing, smart suggestions pop up. If they type "I don't know which component," a list of common components appears.
3. If they have typos, AI fixes them and shows what it changed.
4. User clicks "Analyze with AI."
5. AI comes back with: suggested category, priority, summary, and explains why.
6. If anything's missing, AI asks follow-up questions.
7. If there's a similar open ticket, AI flags it. User decides: link to it or submit separately.
8. User reviews everything, edits whatever they want, and hits Submit.

Path 2: Talk it through (chat)

1. User clicks "Help me describe it."
2. AI opens a chat-style interface and starts asking simple questions one at a time. Like "What type of problem is it?" with options to click.
3. User either picks an answer or types a short reply.
4. AI builds up the ticket as they go. After a few questions, it has enough.
5. The constructed description goes through the same AI analysis as Path 1.
6. User reviews and submits.

Path 3: Call AI support (voice)

1. User clicks "Call AI Support" and picks an accent.
2. AI greets them and says "Take your time, tell me what's going on."
3. User talks as long as they want. Live transcript appears on screen.
4. When they're done, they hit "I'm done talking."
5. User can edit the transcript before AI processes it. This is important because speech recognition makes mistakes, especially with accents or technical terms.
6. AI analyzes the text, asks follow-ups by voice if needed.
7. After 2-3 rounds (or sooner if AI has enough), it wraps up and creates the ticket.
8. User reviews and submits.

Key decision points

These are the moments where the flow branches based on inputs:

Is the description too vague?

If it is (like "my computer is acting weird"), AI doesn't just guess. It switches to guided questions to collect the basics: what, when, who's affected, how bad.

Is this a duplicate?

Before submission, AI checks for similar open tickets. If it finds one, the user chooses: "Yes, link mine to that one" or "No, mine is different." Follows the three-tier privacy model (own tickets: full details, same company: anonymized, cross-company: never shown).

Is the user actually asking about an existing ticket?

If someone says "I submitted a ticket yesterday and haven't heard back," that's not a new issue. AI finds the existing ticket and shows status, assignment, wait time. The user can add a note or request escalation instead of creating a duplicate.

Is this a known issue?

If the user mentions a version upgrade, AI cross-references the known issue database. If there's a match, it shows the article first. The user might not need a new ticket at all.

Is this Severity 1?

If AI detects "system down" + "business impacted" + "everyone affected," it auto-marks Severity 1 and triggers escalation. This one doesn't wait for the user to choose a priority level.

e. Functional Requirements

P0: Can't ship without these

Natural language input. The user types in their own words. No rigid form that forces them to pick a category before they've even explained the problem. The form comes after AI understands what they're saying, not before.

AI suggests category, priority, and summary. Based on what the user said, AI fills in the category and subcategory, suggests a priority, and generates a short summary. Every suggestion comes with a reason so the user knows why. All of this is editable.

AI generates a structured description when the user can't. If someone says "it crashed and I don't know what happened" (UC1, UC7), AI pieces together whatever they said into something an engineer can work with.

Guided intake for users who don't know what to say. When the description is too short or too vague (UC5), AI switches to a step-by-step mode. Simple questions with options to click. Builds the ticket as they answer.

Guided root-cause diagnosis. When someone says "it's not working" and it could be five different things (UC2, UC8), AI asks targeted questions to figure out if it's an access issue, a config problem, or a bug. That determines which team gets it.

Spell correction. AI fixes typos before analyzing. Shows the user what got changed. The corrected version is what AI works with, but the original is kept.

Smart suggestions while typing. If the user types something like "I don't know which component," common components pop up. Same for errors, login issues, performance problems. Helps the user say what they mean without knowing the right terms.

Nothing gets submitted without the user confirming. Every AI suggestion is just a suggestion. The user can change anything. Submit only happens when they click Submit.

P1: Important, ship with v1 or right after

Severity assessment (UC3). This one matters because users report "it's slow" all the time and they have no idea if it's a big deal or not. AI needs to tell them: "This looks moderate, not an emergency, but it should be looked at today." Without this, every performance complaint either gets ignored or gets treated as urgent. Neither is good.

Existing ticket detection (UC4). People come back and say "where's my ticket?" more than you'd think. If AI can just find the old ticket and show them what's going on, that's one less duplicate in the system and one less frustrated customer. The tricky part is detecting that they're asking about an old ticket vs. reporting a new problem. We handle that through tone and keyword analysis.

Known issue and KCS matching (UC6). When somebody upgrades and stuff breaks, there's a decent chance it's already a known issue with a workaround. If AI can surface that before the user creates a ticket, everyone wins. The user gets a fix faster, and the support team doesn't get another ticket for something they already know about.

Log and error parsing (UC7). Deploy failures generate walls of logs. Nobody wants to read through all of that. AI pulls out the part that actually matters so the user can just paste the relevant bit into the ticket. This is a time-saver for both the user and the engineer who picks it up later.

Multi-component breakdown (UC9). Some problems don't belong to one team. Network is flaky, storage is slow, monitoring stopped updating, all at the same time. AI's job here is to figure out which pieces go where, split them up, or just send the whole thing to the SE manager and say "this needs someone senior to look at it."

Duplicate detection. This is where the privacy model comes in. We talked about this: same user sees everything, same company sees anonymized info, cross-company sees nothing. The point is to reduce redundant tickets without leaking anyone's data. It's a fine line but it's important to get right.

Critical incident auto-escalation (UC10). If it's a full outage and the business is stopped, we don't ask the user what priority they want. AI marks Severity 1 and triggers escalation immediately. This is the one case where AI acts first and asks questions later, because every minute counts.

P2: Nice to have, probably v2

Ticket templates. For things like password resets or new user setups, you don't need AI to walk you through it. Just a quick template you fill in and submit. Simple stuff shouldn't require a conversation.

Attachment upload. Right now you can only type. But sometimes a screenshot explains the problem better than three paragraphs. v2 would add file uploads, and eventually pair it with OCR so you can just paste a screenshot and AI reads it.

Historical context. If somebody submitted the same type of ticket three times this month, AI should probably mention that. "Hey, you've reported this before. Want to reference your previous ticket?" Cuts down on repeats.

Multi-issue splitting. People dump three problems into one ticket all the time. "My email is broken AND my printer isn't working AND the VPN is slow." That's three different teams. AI should catch it and offer to split it up.

Sentiment-driven escalation. This would connect to MSPbots' Sentiment Max. If the customer's tone is getting progressively angrier across multiple interactions, flag it before it turns into a churn risk. Not a v1 priority, but a smart thing to build later.

Three topics

1. Required field handling

A ticket can't be submitted without these fields having a value:

- **Product name** (which product is this about)
- **Product version** (what version are they running)
- **Severity** (how bad is it)
- **Component** (which part of the system)
- **Customer basic info** (name, email, company. Usually auto-filled from login)
- **Impact scope** (how many people are affected)

Some of these the user will know. Some they won't. That's fine.

If a required field is missing, AI doesn't just show a red "please complete all fields" warning. Instead it prompts the user with clickable options for each missing field. If the user genuinely doesn't know (like they have no idea which component is broken), they can pick "I don't know" and that's a valid answer. It gets passed along to the support engineer or manager who will figure out the routing from there.

The point is: every required field gets addressed before submission, but we don't block people just because they can't answer everything. "I don't know" is better than leaving it blank and having no one realize it's missing.

2. Incomplete / ambiguous ticket scenarios

"Incomplete" and "ambiguous" are two different problems.

Incomplete means required fields are missing. That's handled above. The system catches it and asks.

Ambiguous is harder. It means the fields technically have values, but the values are useless. Some examples:

- The description is three words: "help me please." There's text in the box but an engineer can't do anything with it.
- The user wrote a whole paragraph but it's all frustration: "I'm so sick of this, nothing ever works, this is ridiculous." Lots of words, zero information about what's actually broken.
- The user picked a category but chose "Other / I don't know" for everything. Fields aren't empty, but they can't be used for routing.

So AI needs to judge not just "is the field filled" but "is the content useful." Here's how we think about it:

What triggers "mildly vague": The description is short (under ~20 words) but has some signal in it, like a product name or an action ("Outlook crashed"). AI asks one or two targeted follow-ups: "When did this start?" "Is it just you or others too?"

What triggers "very vague": The description is extremely short (under ~5 words), or it's longer but has no actionable content (pure emotion, no facts), or the user picked "I don't know" for more than two required fields. AI switches to full guided mode with step-by-step questions.

What triggers "needs a human": If after guided questions the user still can't provide enough for AI to classify, the ticket gets created with whatever we have and flagged for a support engineer or

manager to review manually. We don't block the user forever. At some point you just have to let it through and let a human figure it out.

3. AI-assisted inputs: where AI helps vs. where it must not auto-act

What AI can do:

- Suggest category, priority, summary
- Generate a description from vague input
- Ask follow-up questions
- Flag duplicates
- Recommend severity
- Show known issues

What AI must never do:

- Submit a ticket without the user confirming
- Override a choice the user explicitly made
- Merge or close tickets on its own
- Make assumptions about security-sensitive issues
- Show one company's ticket data to another company

f. Non-Functional Considerations

Data Quality

- AI suggestions must improve ticket quality, not make it worse. Every AI-filled field must be editable.
- We track AI suggestion acceptance rate as a quality signal. If users are rejecting or editing more than 40% of suggestions, something is off with the prompts and we need to look into it. 60% acceptance is the initial target. We'll adjust this based on what we see in Beta.
- The real test of quality: does the engineer still have to go back and ask the customer a bunch of questions after the ticket is created? If AI Intake is working, the number of follow-up rounds should go down. If it doesn't, the AI isn't actually helping.

Trust & Explainability

- Every AI suggestion shows a reason right next to it. In the prototype, there's a purple "AI Suggested" badge on each field with a short explanation underneath, like "Routed to Email > Client App because you mentioned Outlook crashing." Users can see why AI made that call and decide if it's right.
- Every suggestion is editable. The category, priority, summary are all dropdowns and input fields the user can change. AI doesn't lock anything. If you don't agree with what AI picked, just change it.
- When AI asks a follow-up question, it should be clear why it's asking. If AI says "How many users are affected?" the user should understand that's because the answer changes the priority, not just because there's an empty field somewhere.

Response Speed

- AI analysis should feel fast. In the prototype, there's a 1.5-2 second delay after the user clicks "Analyze" before results appear. That's intentional. Fast enough that it doesn't feel slow, but not instant either, because instant feels like AI didn't actually think about it. For production, target is under 3 seconds for the full analysis (category + priority + summary + duplicate check).

Security & Privacy

- Production should use enterprise-grade AI infrastructure (like Azure OpenAI) consistent with MSPbots' existing security setup. The prototype uses mock data, but the production version needs proper data handling.
- Zero data retention for model training. Customer ticket data must not be used to improve models for other customers.
- Tickets can contain sensitive info like passwords, IP addresses, or personal data. AI processing needs to meet the same security and compliance standards MSPbots already follows.
- Strict tenant isolation. AI must never surface one MSP client's data to another. This is non-negotiable.

Duplicate Detection Privacy Model

Duplicate detection operates under a three-tier privacy model:

Tier	Scenario	What AI shows	What AI hides
Same user	User submits a ticket similar to one they already filed	Full ticket details (ID, title, status, description)	Nothing, it's their own data
Same company	Another user at the same company filed a similar ticket	Ticket ID, anonymized title, similarity score, creation time	Submitter name, submitter email, ticket description details
Cross-company	A user at a different MSP client filed a similar ticket	Nothing. Never surfaced to the end user	Everything; may be used internally to link to Known Issues only

This design ensures that duplicate detection provides operational value (reducing redundant tickets, faster resolution through linked reports) without creating privacy risks across organizational boundaries.

g. Success Metrics

Metric	Dimension	Target	How to Measure
AI Intake weekly active usage	Adoption	Majority of portal tickets should go through AI Intake. Exact target set after Beta.	Usage logs / tickets with AI Intake flag
Ticket information completeness rate	Quality	Significant improvement. Measured in Beta.	Check how many tickets get sent back to the customer because info was missing. If that number drops, the intake is working.
Support engineer / manager workload on intake	Efficiency	Should go down. AI handles the intake grunt work (classifying, collecting missing info, routing) so engineers and	Track how many tickets need manual intervention from SE or manager before entering the work queue

Metric	Dimension	Target	How to Measure
		managers spend less time on it.	
AI suggestion acceptance rate	Quality	More accepted than rejected. If users are editing most suggestions, the prompts need work.	Track accept/edit/reject per field
First contact resolution rate	Efficiency	Should improve. Exact target TBD after Beta.	Check how many tickets the engineer can start troubleshooting right away vs. how many they have to ask the customer for more info first.
Median case resolution time	Efficiency	Measurable reduction. This is the one that matters most to the business.	Track median time from ticket creation to resolution. Median instead of average so a few outlier tickets don't skew the number.
Median time on ticket creation page	Efficiency	Noticeable reduction. If AI is helping, users shouldn't need as long to figure out what to fill in.	Track time from page open to submit

h. Risks & Open Questions

Unknowns

Things we don't have answers for yet. We need real users to find out.

- **Which intake mode will people actually use?** We built three (type, chat, voice) but we have no idea which one gets the most usage. Maybe everyone just types. Maybe non-technical users love the chat mode but technical users ignore it. We won't know until it's out there.

- **How accurate is AI classification in the real world?** The prototype uses mock data. Real tickets are messy, full of typos, slang, half-sentences. Accuracy could be 90% or it could be 60%. We need real data.
- **How different are MSP environments from each other?** Every MSP uses different products with different component names and categories. Will a general-purpose prompt work for most of them, or will it break for anyone who doesn't use standard naming?
- **Will users actually talk to an AI on a voice call?** It sounds cool in a demo but some people might find it weird or uncomfortable to talk out loud about their IT problems. Especially in an open office.

Assumptions we're making (need to validate)

- We assume users are willing to interact with AI instead of just filling out a form. Maybe they'd rather have a simple form with fewer fields.
- We assume AI's suggested classification is more accurate than what users would pick themselves. It might not be. Users who know their system well might be better at classifying than AI.
- We assume better ticket information leads to faster resolution. It sounds obvious, but we haven't measured it. It's possible that resolution time depends more on engineer availability or problem difficulty than on ticket quality.
- We assume duplicate detection helps more than it annoys. If it flags too many false positives ("this looks like a duplicate" when it's not), users will start ignoring it or get frustrated.
- We assume users will answer AI's questions honestly. But some people just want to submit and move on. They'll pick whatever gets them through fastest. When someone clicks "I don't know" in the guided flow, they might actually know but can't be bothered to think about it.
- We assume the three modes we built cover what users need. But maybe some people just want a dead-simple form with three fields, no AI, no guidance, no conversation. We didn't build that option.

How we plan to validate

1. **Track data from day one.** Which mode people pick, how long they spend, whether they change AI suggestions, how many tickets get sent back for more info. Without this data we're guessing.
2. **Talk to actual users every week.** Not surveys, real conversations. Ask coordinators and engineers straight up: are the tickets better? Are you spending less time chasing info? Numbers show what happened. People tell you why.
3. **Run shadow mode before launch.** Users submit tickets the old way, AI silently runs in the background. Compare AI's output vs what the user actually filled in. We know accuracy before

anyone sees it.

4. **Have engineers rate tickets.** Grab 50 random tickets a week. Ask: could you start troubleshooting right away, or did you need to go back for more info? Compare AI-assisted vs non-assisted.
5. **Monitor the engineer's first action.** When they open an AI-assisted ticket, do they start troubleshooting or message the customer for more details? If it's still "ask for more info," AI Intake didn't solve the problem.
6. **Track mode switching.** Someone starts typing then switches to guided chat? Typing wasn't enough. Switches from guided back to typing? Too many questions.
7. **Talk to people who quit.** If a Beta user stops using it or turns off a feature, find out why. People who leave tell you more than people who stay.
8. **Kill what doesn't work.** Nobody uses voice call? Cut it. Duplicate detection more annoying than helpful? Make it optional. Beta is for learning what works, not defending what we built.